Kubernetes

Probeで
ヘルスチェックを自動化する

Readiness / Liveness / Startup の使い分けと設計指針

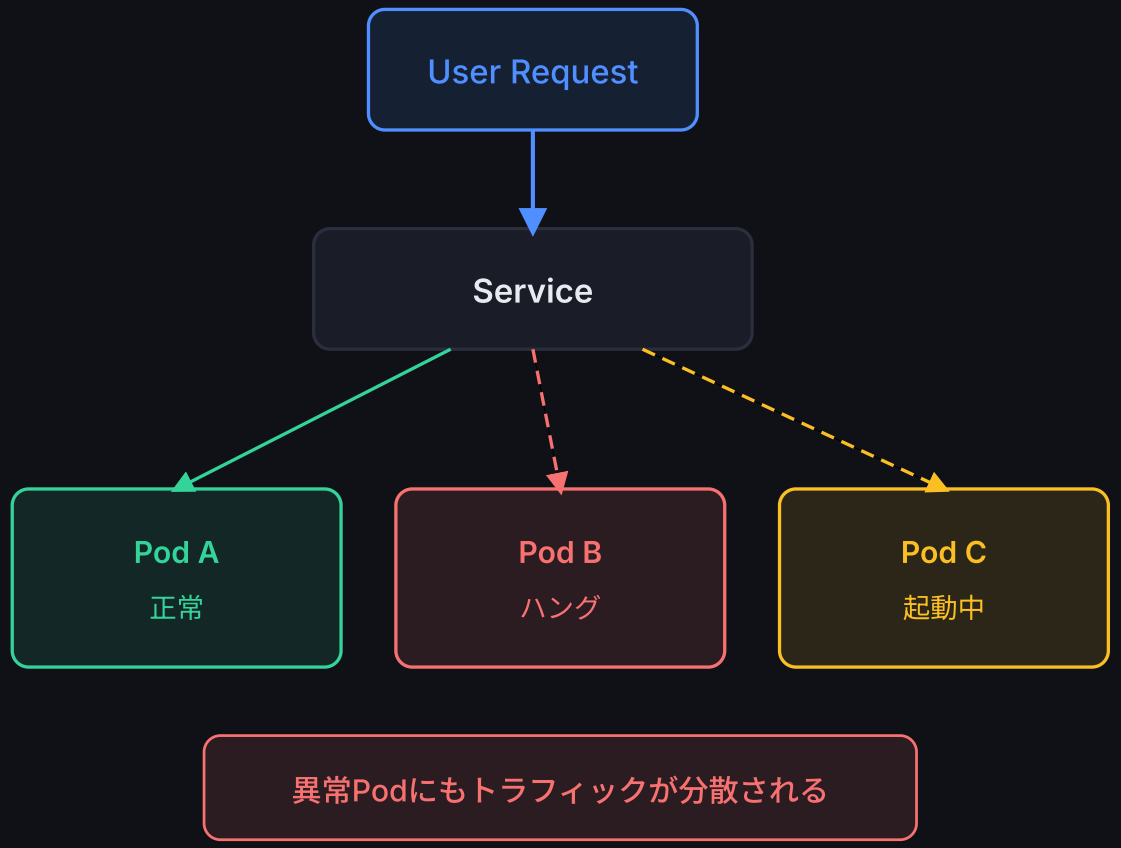
viserhaut — 2026

SECTION 01

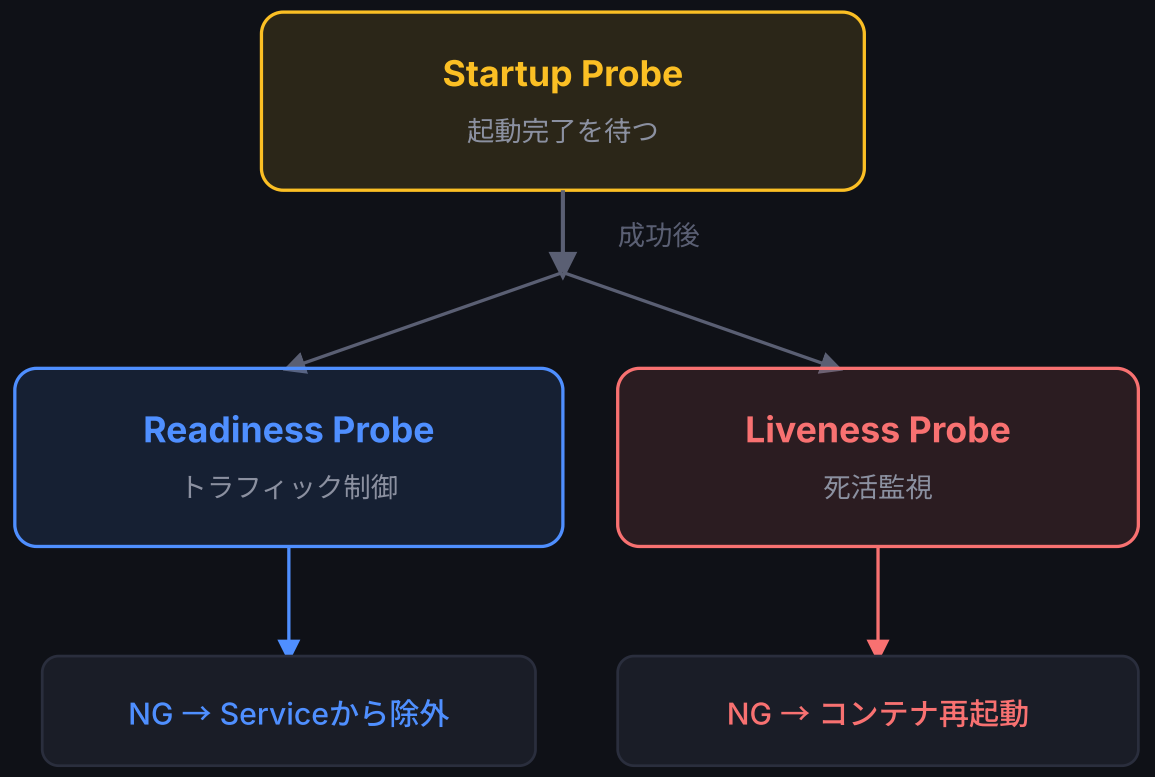
Probeが解決する問題

Probeなしではコンテナ障害がユーザーに直撃する

- **異常Podへリクエスト転送**が続き、ユーザーに**500エラー**が返る
- プロセスがハングしても**再起動されず**、障害が長期化する
- 起動途中のPodにトラフィックが流れ、**初期化エラー**が発生する



3種のProbeがそれぞれ異なるレイヤーを守る



- Readiness Probeが失敗すると、Serviceのエンドポイントから**除外**される
- Liveness Probeが失敗すると、kubeletがコンテナを**自動再起動**する
- Startup Probeが完了するまで、他のProbeは**実行されない**

Readiness vs Liveness vs Startup — 判断基準の比較

観点	READINESS	LIVENESS	STARTUP
目的	リクエスト受付可能か	プロセスが生きているか	起動完了を待つ
失敗時の動作	Serviceから除外	コンテナ再起動	コンテナ再起動
典型的な用途	DB接続待ち、依存先未応答	ハング、メモリリーク	Java等の起動が遅いアプリ

本番APIサーバーのProbe設定例

```yaml
containers:
- name: api
  livenessProbe:
    httpGet:
      path: /api/health
      port: 8080
    initialDelaySeconds: 15
    periodSeconds: 10
    failureThreshold: 3
  readinessProbe:
    httpGet:
      path: /api/health
      port: 8080
    initialDelaySeconds: 5
    periodSeconds: 10
    failureThreshold: 3
```

liveness
initialDelay=15sで起動完了を待ってから死活監視を開始

readiness
initialDelay=5sで早期にトラフィック制御を開始

failureThreshold
3回連続失敗で判定。一時的なスパイクで誤判定しない

Probe判定フローとPodライフサイクル

本番運用でのProbe設計3原則

01

Readiness + Livenessは必ずセットで設定する

異常Podへのトラフィック遮断と自動再起動の両方がないと、障害影響が拡大する

02

起動時間が読めないならStartup Probeを前段に置く

initialDelaySecondsだけでは無駄な待ちが生じる。Startup Probeなら起動直後に次へ進める

03

failureThresholdで一時スパイクを吸収する

1回の失敗で即再起動すると、負荷スパイク時に全Podが再起動ループに陥るリスクがある